Department of Computer Science and Engineering
Tandon School of Engineering
New York University

Homework 2 : x86 Bare Metal Exercises

CS-UY 3224, Intro to Operating Systems

Abstract

This homework is designed to familiarize you with the layout and constraints of a bootloader for an x86 machine. It will also familiarize you with basic x86 assembly conventions and syntax. Finally, you will use BIOS interrupt services to implement the some simple programs in a real mode x86 environment.

Academic Integrity

All work submitted for this assignment must be your own. Cheating and plagiarism will not be tolerated. NYU Tandon's policy on academic misconduct[1] applies to this assignment. If you have questions about appropriate conduct with regards to homework that are not answered by the syllabus or Tandon's code of conduct, please contact your professor.

Introduction

The classic IBM PC boot sequence is preserved unaltered in many of the machines you interact with today. When a machine's firmware has performed minimum harware initialization and power-on self tests, it will try to hand over control to software located on a storage peripheral.

Today there exist two methods and interfaces for this handoff to occur. The modern system is the Unified Extensible Firmware Interface (UEFI), while the legacy system is the Basic Input/Output System (BIOS). In this assignment we will use the BIOS to write some simple programs against "bare metal" x86, no operating system services or user library provided.

When using the BIOS, the machine's firmware will search for special 512 byte sectors at the beginning of attached storage peripherals. These sectors are called **boot sectors** and are signified by a **boot signature** which consists of two bytes, 0x55 and 0xFF, at the final two addresses in the sector, 0x1FE and 0x1FF respectively.

When a boot sector is found, the firmware will load the 512 bytes into device memory at address 0x7C00[2] and then transfer execution to that address. When this transfer happens, the CPU will be operating in 16-bit **real mode**, an environment that is backward compatible with and nearly identical to the original 8086 processor.

The programs you write for this assignment will be valid BIOS boot sectors and will be operating in this real mode environment.

[1] NYU Tandon Student Code of Conduct

[2] The original IBM PC required at least 32KB of memory to load an OS from a floppy drive. The decision was made to load the bootloader at the end of this available memory space. 0x7C00 is 31KB; of the remaining 1KB, half was for the boot sector, and the other half was for the bootloader's stack or other memory uses.

Before You Start

For all of the following exercises, you DO NOT need to add the paddings and the magic bytes yourself like what we went over in class. The assignment is set up such that the build process the paddings and the **boot signature** are automatically included when you run `make`.

If you are getting errors like:

```
ld -melf_i386 -T link.ld 1.o -o 1.elf
ld: section .sig LMA [0000000000007dfe,0000000000007dff] overlaps
    section .text LMA [0000000000007c00,0000000000007dff]
make: *** [Makefile:40: 1.elf] Error 1
```

It is likely that your program exceeds the 512 bytes limit in length due to the padding you add or the overall verbosity of your program.

Part 1: Assembly refresher

First of all, in the Intel assembly language, it is configured that:

- representing a hexadecimal number `1a`, which is $1 \times 16^1 + 10 \times 16^0 = 26$ in decimal, comes in two ways:

  (1) suffixing an `h` to the hex number: `1ah`

  (2) prefixing `0x` to the hex number: `0x1a`

- Capitalization of assembly code does not make a difference except in strings.

Answer the following questions in `ans.md`.

1. In a 16-bit x86 system, how many bits can `cx`, `cl`, `ch` store, respectively?
2. In a 32-bit x86 system, how many bits can `cx`, `cl`, `ch`, `ecx` store, respectively?
3. 1 byte = _____ bits
4. In an x86 system, how many bytes are needed to represent an ASCII character?
5. In a 16-bit x86 system, among `cx`, `cl`, and `ch` what's the smallest register needed if we would like to carry one ASCII character? What if to carry a decimal number `95`? Choose the appropriate option for each question.

   A. `ch` or `cl`

   B. `cx`
6. At least how many bytes are needed to represent the hexadecimal number `0x67`?
7. What letter does the number `0x67` represent in ASCII?
8. After running the following instructions, what is in `ah`, `al`, `bh`, `cx`, `dh`, `dl`, respectively? Write your answer in decimal. You may assume that the registers are set to 0 initially.

```
xor dx, dx
mov ax, 1
mov bh, 4bh
mov al, bh
mov cx, 0x000b
div cx
```

For the rest of the parts, you will write assembly programs in 16-bit x86 system.

Part 2: Print a character

The goal of this part is to write your first 16-bit bare-metal program, which prints a single character.

Two versions of **2.s** are provided, **intel/2-starter.s** and **att/2-starter.s**. Copy one of them outside into a file named **2.s** depending on your prefered syntax. The syntax used in Computer Architecture is the Intel syntax, and the syntax used in the Operating Systems slides and xv6 source code is AT&T syntax.

If you want to use AT&T syntax, set the first line of the Makefile to: `USE_NASM = false`

`int 10h` or `int 16` is a shorthand for BIOS interrupt call 10hex in an x86-based computer system. Typically, the BIOS provides video services by setting up a real-mode interrupt handler. Such services include setting the video mode, character and string output, and reading and writing pixels in graphics mode. `int` stands for "interrupt", and the hexadecimal number `10h` (or `16` in decimal form) means this is the 17th interrupt vector in an x86-based system. (Wikipedia)

Generally, to use this call, it's expected to load a function code in specifying which function to use in `ah` register. Then load other required parameters in other registers, and make the call. A list of functions, function code, function parameters and return values are listed in this table on Wikipedia.

Among many function services, we will practice the "Teletype output" function:

Function	Function code	Parameters	Return
Teletype output	`ah=0eh`	`al` = Character	None

(Wikipedia list `bh` and `bl` as function parameters, while we don't use them in our simple system, so ignore them in the table as well as in this assignment)

`0x0E` (or `0Eh`) is a hexadecimal number chosen to represent the interrupt index of "teletype output", which "writes a character (to the screen)". This interrupt prints the ASCII character stored in `al`. In a nutshell, printing a `single` character can be summarized into a three-step subroutine:

- Step 1: Move the character we would like to print to `al`

  ```
  mov al, `e`
  ```

  Two backticks symbols (`` ` ``) wrap a character or characters in between. What's inside the backticks is case-sentitive. So `` `e` `` and `` `E` `` is encoded differently after compiled.

  Alternatively, we can move the ASCII number of the character we would like to print into `al`:

  ```
  mov al, 0x65
  ```

- Step 2: Move the function code `0x0e` into register `ah`, specifying "Teletype output" function to use for `int 10h` interrupt handler.

  ```
  mov ah, 0x0e
  ```

- Step 3: Call the interupt handler

  `int 10h`

Note that step 1 and step 2 are interchangeable, while step 3 must be the last because you really need to prepare the function parameters before calling the function.

Now, fill in `2.s` with the three-step subroutine to print the greater-than sign (`>`). Your code should contain 3 instructions exactly.

You can test your program with the make command: `make qemu-2`.

Complete `2.s` (copy from the starter of your selected syntax and remove `-starter` from its name) for this part.

Part 3: Print from memory

The goal of this part is to practice with loop and memory in assembly.

Borrow your experience from printing one character in part 2, and write a program that prints "Hello World" to the screen. It's recommended to use `loop` and `lodsb` instructions:

- Syntax of `loop`: `loop [LABEL]`

  The `loop` instruction is essentially a shortcut to several instructions in sequence: it decrements the value of `cx` and compares it with zero. If the value in `cx` is not equal to zero, the program jumps to the specified label. Otherwise, the program exits the loop.

  `loop loop_start` is equivalent to

  ```
  dec cx
  jnz loop_start
  ```

- Syntax of `lodsb`: `lodsb`

  `lodsb` stands for "load string byte". This instruction does not take explicit operands like `mov` and `loop` do. It is used for loading a byte from the memory location pointed to by the source index (`si`) register into the `al` (accumulator) register. Hence, before you `lodsb`, make sure `si` points to the start of your string buffer.

  `lodsb` is equivalent to

  ```
  mov al, [si]
  inc si
  ```

In the starter code, the string you need to print is predefined with a label `message` marking the beginning of the string.

```
message:
    db `Hello World\r\n`
```

Here `db` stands for "define byte", a directive used to define bytes of data in the memory.

Some basic `db` usage:

- `my_byte db 42 ; Defines a single byte with the value 42`

- `my_string db `Hello, World!`; Defines a sequence of bytes to store the string`
- `data db 10, 20, 30, 40 ; Defines four bytes with values 10, 20, 30, and 40`

The length of the string is also calculated and equated to the symbol `message_len` by subtracting the current address (`$`) from the address of the message label.

```
message_len equ $ - message
```

`equ` is a directive that equates a reference to the symbol on its left to be the same as a reference to the value on its right. If you do

```
mov cx message_len
```

`cx` will then contain a number which is the length of the string.

Notice that the string message we define ends with `\r \n`. `\r` is a carriage return (CR), and `n` is a line feed (LF). The former returns the cursor to the start of the current line, while the latter moves the cursor down a line.[3] See what happens if you remove either of them when printing out the message.

You can test your program with the make command: `make qemu-3`.

Complete `3.s` (copy from the starter of your selected syntax and remove `-starter` from its name) for this part.

Part 4: Echo

The goal of this part is to practice reading user input. On the high level, there are two phases: reading and printing. The program should read arbitrary lengths of user input. When the user presses "Enter" key, the program should stop receiving user input, print a new line, then start to print all the user input. The functional effect of this program looks like the same as the `echo` unix program, and that's why this part is named.

Reading phase One subtlety is to realize, key pressing and showing what was pressed on the screen are two different things. At bootloader, reading what value of the pressed key is handled by a keyboard interrupt handler. This handler is the 23rd interrupt handler with function code `ah=0x00`. This function is to read **one** ASCII character from the keyboard, storing what is read into register `al`. See the following table (source) for details:

Interrupt Handler `int 16h`

Function	Function code	Device	Parameters	Return
Read key press	`ah=0x00`	Keyboard	None	`ah` = Scan code of the key pressed down `al` = ASCII character of the button pressed

After reading what's pressed, there is nothing to be seen on the screen. User does not get feedback at what's pressed at this moment, nor can they confirm whether they pressed is what they want to press - again, pressing a key and displaying what's pressed are two different things. Usually to have a good user experience, immediately after reading a key that user has pressed, you should print it to the screen using our old friend `int 10h`. User can enter arbitrary lengths of input, so there must be a loop.

A side note is that since you will use what you read later on so you should put the user input into the memory.

[3] You might have noticed that xv6 does not require you to end your message with CRLF, because in the Unix family of OSes, a singular LF is conventionally used to represent a newline

When user presses `ENTER`, you should print a carriage return `\r` (`CR`, ASCII `0x0d`) and a line feed `\n` (`LF`, ASCII `0x0a`) to reflect that the user wants to enter new line.

Printing phase Now you should have all the user input in the memory. Print them out. Printing a newline is optional. This phase is practiced in part 3. Once printed, end the program.

Please don't mix up *the printing subroutine within reading phase* and *the printing phase following the reading phase*. The subroutine of printing within reading phase is to reflect what key is pressed, which is done immediately after every key press. The printing phase is to print all the user input together, which is part of the functionality of an Echo program.

Here is a routine outline you may find helpful to follow.

```
read_and_reflect:           ; Reading phase starts
    call read_char          ; Read a character from the keyboard and store to memory
    cmp al, 0xd             ; Check if Enter key is pressed (ASCII 13)
    je end_user_input       ; If Enter is pressed, go to printing phase
    call print_char         ; Print the character to reflect what is pressed
    jmp read_and_reflect

end_user_input:             ; Printing phase starts
    call print_newline      ; Print a new line
    call print_memory       ; Print all user input from memory
    jmp done                ; No need to print a newline again

done: jmp done
```

To store the input into memory, you may use `stosb`:

Syntax of `stosb`: `stosb`.

Similar to `lodsb`, this instruction does not have explicit operands. `stosb` copies the value in `al` into the location pointed to by `es:di`. `di` is then incremented. It is equivalent to

```
; Assuming es:di is set to the destination address
; Assuming al has the value you want to store
mov [es:di], al
inc di
```

Sample interaction in the terminal:

```
Hi, this is Junyao's speaking. User is typing
Hi, this is Junyao's speaking. In a new line, print all user input. Then end the program.
```

You can test your program with the make command: `make qemu-4`.

Complete `4.s` (copy from the starter of your selected syntax and remove `-starter` from its name) for this part.

Part 5: Numbers

Part 5.1

Read **5-1.s** (copy from the starter of your selected syntax and remove `-starter` from its name) and finish the missing comments interpreting what it does.

Complete `5-1.s` for this part.

Part 5.2

The reverse operation of **atoi** is **itoa**. An **itoa** operation converts an integer to ASCII. Write a simplified **itoa** that converts a single-digit non-negative number to a character and prints it out.

You should hardcode the input integer (single-digit) in a register. For testing, we will change it to different values and check the output.

You can test your program with the make command: `make qemu-5-2`.

```
...
mov al, 5  ; the input is a hardcoded integer
call itoa  ; your subroutine that converts it into ASCII
...
```

The expected output will then be:

```
5
```

Complete `5-2.s` (copy from the starter of your selected syntax and remove `-starter` from its name) for this part.

Part 5.3

Expecting the integer to have multiple digits (up to 255), write a program that converts it into a string of ASCII characters, stores the string in memory and prints it out. You might find reusing your code from previous parts helpful.

Same as 5.2, you should hardcode the input integer that takes up to a single byte in a register. For testing, we will change it to different values and check the output.

You can test your program with the make command: `make qemu-5-3`.

```
...
mov al, 108  ; the input is a hardcoded integer
call itoa  ; your subroutine that converts it into ASCII
...
```

The expected output will then be:

```
108
```

Complete `5-3.s` (copy from the starter of your selected syntax and remove `-starter` from its name) for this part.

Part 6: Caesar Encoding

Write a program that does the following in sequence:

1. Read user input. You may assume that there will only be uppercase letters, i.e., no spaces, no numbers, lowercase letters, symbols, etc.

2. Read the `seconds` from CMOS (Complementary Metal-Oxide-Semiconductor) clock. Make sure that you understand how it is implemented in the starter code for part 6. Reading the "seconds" (and other time data) from the CMOS involves interacting with the real-time clock (RTC) chip, which is part of the CMOS chip. The RTC chip keeps track of time, including seconds, minutes, hours, and more. You can access the `seconds` value from the RTC chip by reading the *appropriate* CMOS registers. `seconds` is an 8-bit integer range from 0 to 59. Here's a general outline of the steps to read the `seconds` from the CMOS using x86 assembly:

   (a) Select the RTC register you want to read: In the case of `seconds`, you'll need to read from RTC register 0x00, which is the "Seconds" register.

   (b) Send the register address to the CMOS chip: You can do this by writing the register address to the CMOS Control Register (port 0x70).

   (c) Read the value from the selected register: Read the data from the CMOS Data Register (port 0x71).

3. Get the last digit of the `seconds` you have read. Call this digit `S` and **print it out**. For example, if you have read `42`, only `S=2` will be printed out.

4. Encode the user input by right-shifting each character `S` positions in the ASCII. Wrap up to the front if right shift goes off the letter `Z`. In other words, for each character `C` from the user, the encoded character `E` is found to be `E = (C+S) % 26`. For example, if the user input is `HELLOZ`, shifting every character by `S=1` should give you `IFMMPA`.

5. Print out the encoded string.

Sample interaction:

```
$ make qemu-6
Please enter a string
HELLOZ
The key is: 1
The encoded text: IFMMPA
$ make qemu-6
Please enter a string
ABCDEFGHIJKLMNOPQRSTUVWXYZ
The key is: 5
The encoded text: FGHIJKLMNOPQRSTUVWXYZABCDE
```

Please notice the newline after printing each subtask messages, user inputs and outputs. **You should follow the same format as the sample interaction.** That is to say, print a carriage return and a line feed at the appropriate places. Newline is required after the line that starts with "The encoded text: ..." so that the cursor will go back to the left front after finishing all the tasks.

You can test your program with the make command: `make qemu-6`.

Complete `6.s` (copy from the starter of your selected syntax and remove `-starter` from its name) for this part.

Grading

Partial credits will granted for correct subroutines.

Bugs and other non-functional elements should be documented in your comments. A documented bug with a note of what was done to try to fix it will be penalized less than bugs discovered during grading.

Parts		Points
Part 1	Assembly Refreshers	15
Part 2	Print a character	10
Part 3	Print from memory	10
Part 4	Echo	20
Part 5	Numbers	25
Part 6	Caesar Encoding	20